T.C. İSTANBUL KÜLTÜR ÜNİVERSİTESİ

COM5005 – WEB PROGRAMMING

LAB 11 – HTML Helpers, Form Validation

After completing this Lab, you will be able to

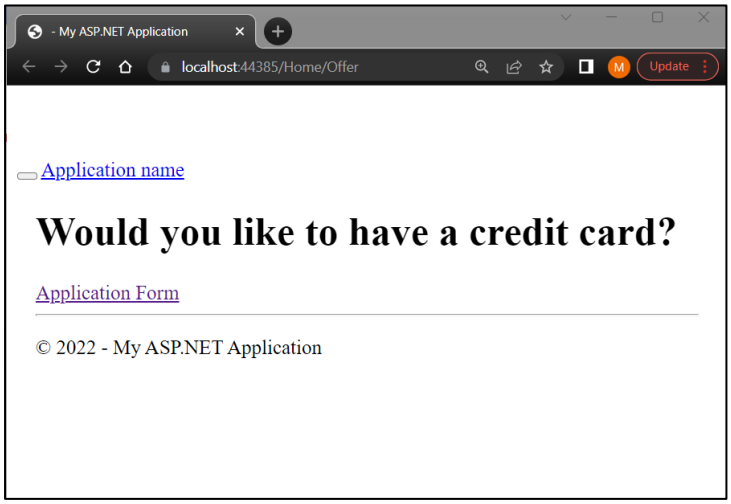
- Create Html Form using @HtmlHelpers
- Create Annotation
- Define HTML Form Validation

PROCEDURE 1 – Creating a New Form and Validation of Data.

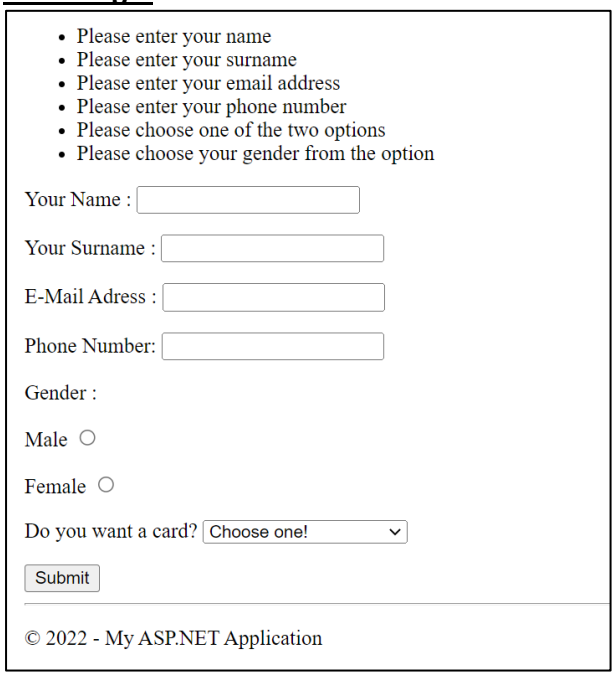
We will create a simple data entry application written in MVC5.

- First, let's create a home page and ask users if they want a credit card.
- Then let's direct users to the credit card application form with a link.
- Finally, let's create a results page showing that the records have been received.

Home Page:



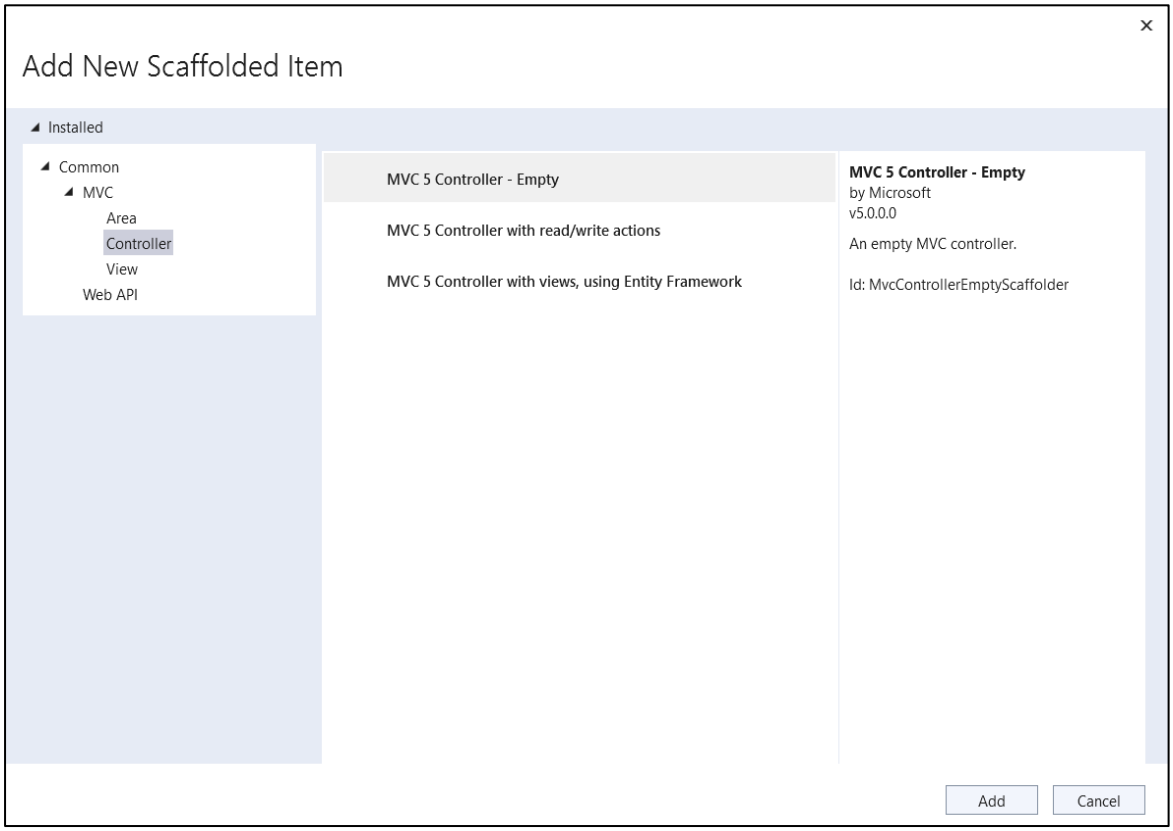
Form Page:



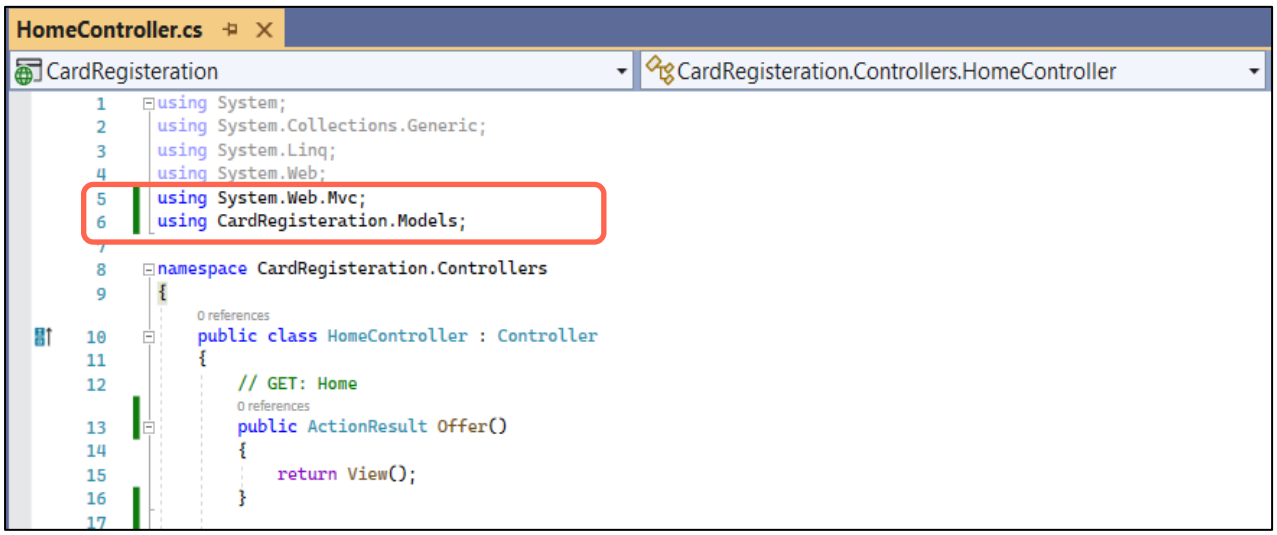
Step 1 – Create a new project. And Select ASP.NET Web Application (.NET Framework) for templates. And then click to Next.

- Name your project on the Configure Your New Project window.
- Select the Empty project and select MVC template.

Step 2 – Right click on the Controller folder and create an empty controller. Name it "**HomeController**".



Step 3 – Let's create an **Offer Action Result**. This method should return the **Offer.cshtml** page.



Step 4 – Let's create our first page called **Offer.cshtml**. This page will be our home page.

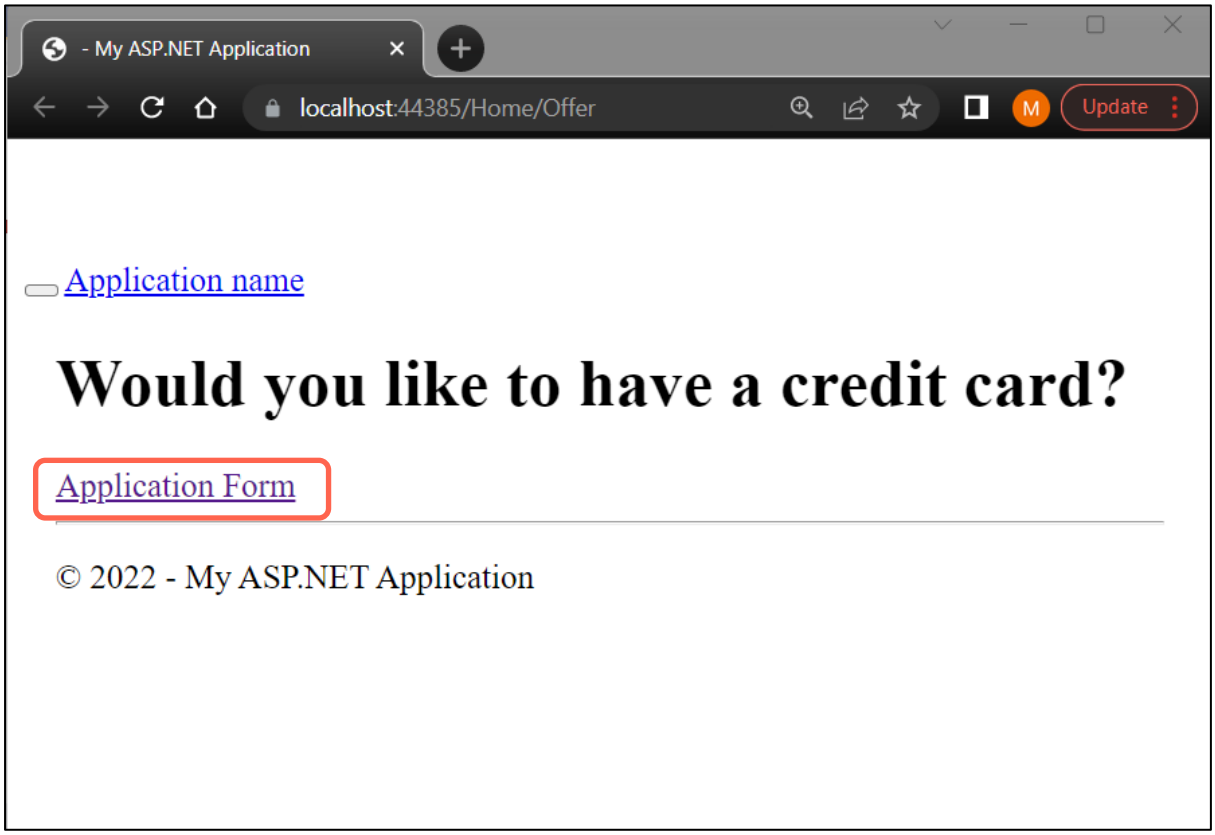
We add an **ActionLink** for the transition from the first page to the **Form.cshtml** web page.

The link appears with the name “Application Form”.

Offer.cshtml

```
1
2   <html>
3   <head>
4       <title>Credit Card Registration</title>
5   </head>
6   <body>
7       <div>
8           <h1>Would you like to have a credit card?</h1>
9           @Html.ActionLink("Application Form", "Form")
10      </div>
11  </body>
12  </html>
13
```

** Execute the code and review the results.



Step 5 – When the “Application Form” link is clicked, the **Form.cshtml** web page will be opened. This page contains all form elements. Form members are created and called with @HtmlHelper.

However, we want to control the validation of the data entered in the form elements. For this, we create a class called Card.cs.

Step 6 – Right click on the **model** folder and create a new class. Name the class **Card.cs.**

An empty model containing the Card class will open. Firstly;

"`using System.ComponentModel.DataAnnotations;`" we add the feature to the model.

Step 7 – We will define some requirements for each form element. Then we will give some **annotation** if these conditions are not met.

Validation conditions were given, such as the requirement to fill in the blanks, choosing one of the optional items, and ensuring that the e-mail address is spelled correctly using **"Regular Expression".**

Card.cs

CardRegisteration | CardRegisteration.Models.Card

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Web;
5  using System.ComponentModel.DataAnnotations;
6
7  namespace CardRegisteration.Models
8  {
       1 reference
9      public class Card
10     {
11         [Required(ErrorMessage = "Please enter your name")]
           0 references
12         public string Name { get; set; }
13
14         [Required(ErrorMessage = "Please enter your surname")]
           0 references
15         public string SurName { get; set; }
16
17         [Required(ErrorMessage = "Please enter your email address")]
18         [RegularExpression(".+\\@.+\\..+",
19         ErrorMessage = "Please enter a correct address")]
           0 references
20         public string Email { get; set; }
21
22         [Required(ErrorMessage = "Please enter your phone number")]
           0 references
23         public string Phone { get; set; }
24
25         [Required(ErrorMessage = "Please choose one of the two options")]
           0 references
26         public bool? Choice { get; set; }
27
28         [Required(ErrorMessage = "Please choose your gender from the option")]
           0 references
29         public string Gender { get; set; }
30     }
31 }
```

Step 8 – Now let's add controls and then views for the design of the Form.cshtml web page.

Step 9 – Let's bind the Model class into the HomeController.

"`using CardRegisteration.Models;`"

We will use the object that we will derive from this class in the Form ActionResult method.

```
1   using System;
2   using System.Collections.Generic;
3   using System.Linq;
4   using System.Web;
5   using System.Web.Mvc;
6   using CardRegisteration.Models;
7
8   namespace CardRegisteration.Controllers
9   {
        0 references
10      public class HomeController : Controller
11      {
12          // GET: Home
            0 references
13          public ActionResult Offer()
14          {
15              return View();
16          }
17
18          [HttpGet]
            0 references
19          public ActionResult Form()
20          {
21              return View();
22          }
23          [HttpPost]
            0 references
24          public ActionResult Form(Card crd)
25          {
26              if(ModelState.IsValid)
27              {
28                  ViewBag.name = crd.Name;
29                  ViewBag.surname = crd.SurName;
30                  ViewBag.email = crd.Email;
31                  ViewBag.phone = crd.Phone;
32                  ViewBag.gender = crd.Gender;
33
34                  return View("Result", crd);
35              }
36              else { return View(); }
37          }
38      }
39  }
```

The ModelState.IsValid Attribute is associated with HttpPost. Thanks to this feature, we can use it to return to the form page if the form is entered in a way we do not want.

If the entire form is filled in correctly, it will return the Result.cshtml page, otherwise it will remain on the Form page.

It displays the **data** it receives from the form elements on the **Result.cshtml** page with the **ViewBag**.

Step 10 – Creating form elements using @HTMLHelpers.

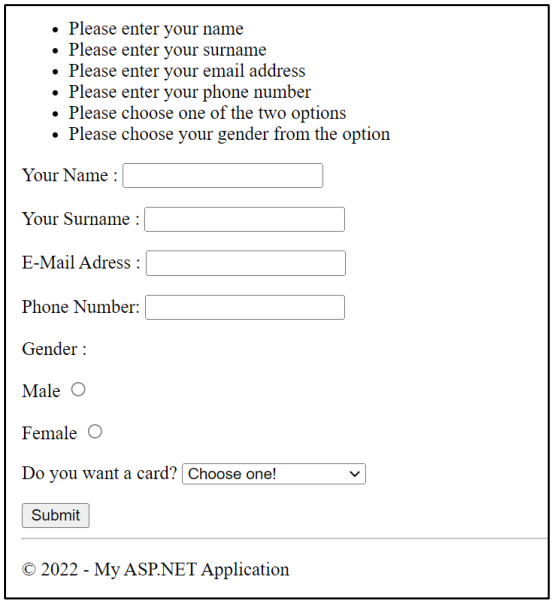
In our form, we have arranged the name, surname, e-mail, phone parts as **textbox**, gender as **radiobutton** and preference part as **dropdownlist**.

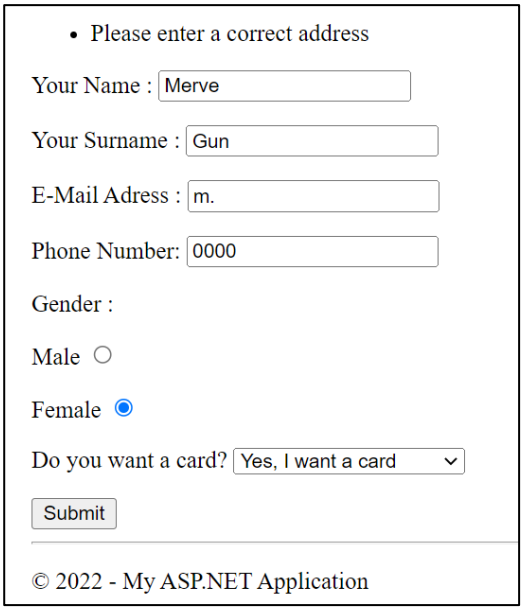
Form.cshtml :

Form.cshtml

```
@model CardRegisteration.Models.Card

<html>
<head>
    <title>Credit Card Registration</title>
</head>
<body>
    @using (Html.BeginForm())
    {
        @Html.ValidationSummary(){
        <p>Your Name        : @Html.TextBoxFor(x => x.Name)</p>
        <p>Your Surname     : @Html.TextBoxFor(x => x.SurName)</p>
        <p>E-Mail Adress    : @Html.TextBoxFor(x => x.Email)</p>
        <p>Phone Number: @Html.TextBoxFor(x => x.Phone)</p>
        <p>Gender : </p>
        <p>Male @Html.RadioButton("Gender", "Male")</p>
        <p>Female @Html.RadioButton("Gender", "Female")</p>
        }
        <p>
            Do you want a card?
            @Html.DropDownListFor(x => x.Choice, new[]
            {
                new SelectListItem() {Text = "Yes, I want a card", Value = bool.TrueString},
                new SelectListItem() {Text = "No, I don't want a card", Value = bool.FalseString}
            }, "Choose one!")
        </p>


        <input type="submit" value="Submit" />
        <link rel="Stylesheet" href="@Href("~/Content/Site.css")" type="text/css" />
    }
</body>
</html>
```

** Execute the code and review the results.





Step 11 – If the form data is Submit, the **Result.cshtml** page will open.

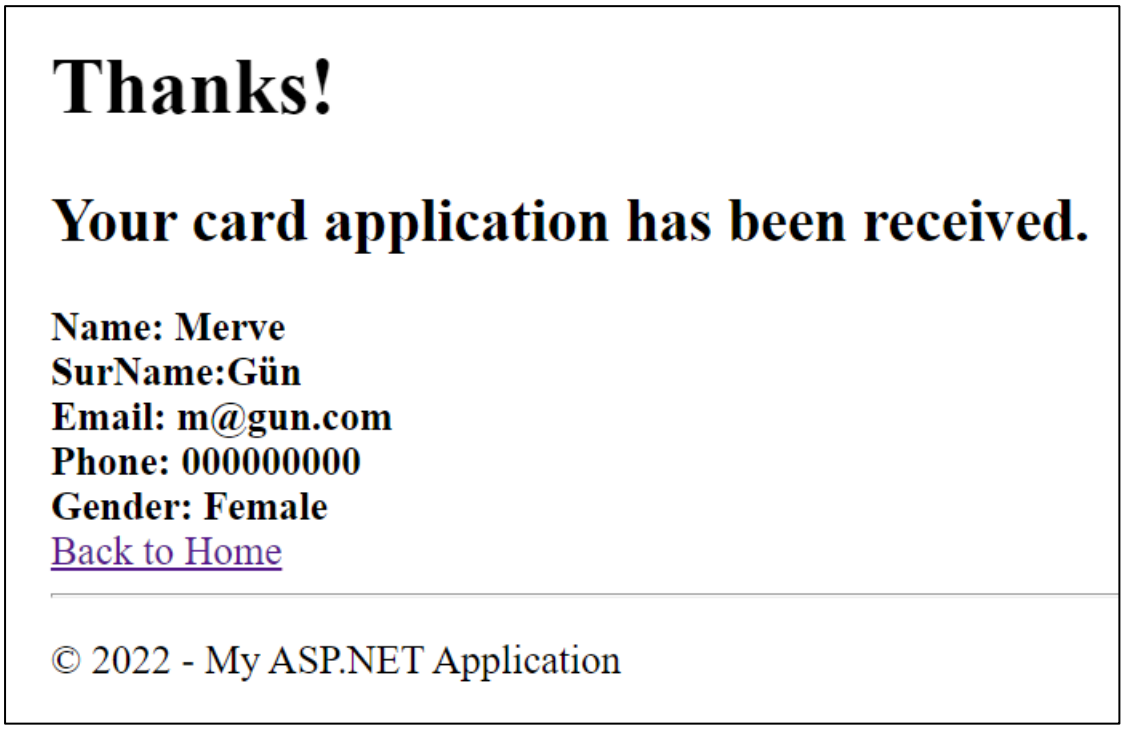
Result.cshtml page consists of 2 parts. The first option will show all the form data if the card request has been made. The second option is the content to display if no card is requested.

Result.cshtml

```
@model CardRegisteration.Models.Card

<html>
<head>
    <title>Sonuc</title>
</head>
<body>
    <div>
        @if (Model.Choice == true)
        {
            <h1>Thanks!</h1>
            <h2>Your card application has been received.</h2>

            <b>
                Name:    @ViewBag.name<br />
                SurName:@ViewBag.surname<br />
                Email:   @ViewBag.email<br />
                Phone:   @ViewBag.phone<br />
                Gender: @ViewBag.gender<br />
            </b>
            @Html.ActionLink("Back to Home", "Offer")
        }
        else
        {
            <h1>Thanks!</h1>
            <h2>You don't have a card application.</h2>
            @Html.ActionLink("Back to Home", "Offer")


        }

    </div>
</body>
</html>
```

** Execute the code and review the results.

First Option -True:



Second Option -False: